Ownership & Borrowing in Rust

A small bank, from Python to Rust

Florian Walter & Anto Grgic

UZH · Programming Languages and Paradigms
11.11.2026

What we'll do

- Python bank → Rust, live
- You vote: **Compiles?** ✋
- We swap roles halfway

The Python bank

```
python3 python/bank.py
```

A value has one owner. Passing it moves it.

Part 1: naive port

```
fn print_report(accounts: Vec<Account>) { .. }
print_report(accounts);
print_report(accounts);
```

Part 1: naive port

```
fn print_report(accounts: Vec<Account>) { .. }
print_report(accounts);
print_report(accounts);
```

```
error[E0382]: use of moved value: `accounts`
```

The first call moved `accounts`.

Part 1: the fix

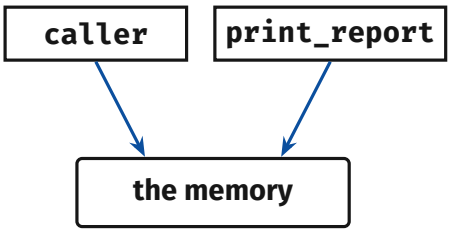
```
fn print_report(accounts: &[Account]) { .. }
print_report(&accounts);
print_report(&accounts);
```

Borrow with &.

Part 1: C, double free

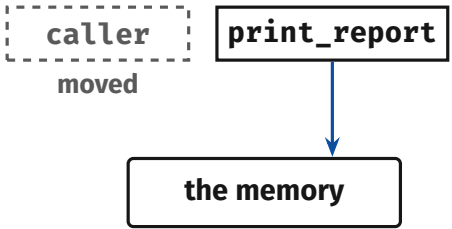
what could go wrong

C: pass a pointer



both call `free`: double free

Rust: pass ownership



one owner: freed once

Changing data needs `&mut`, and only one `&mut` at a time.

Part 2: naive port

```
fn transfer(src: &Account, dst: &Account, amount: i64) {
    src.balance -= amount;
    dst.balance += amount;
}
```

Part 2: naive port

```
fn transfer(src: &Account, dst: &Account, amount: i64) {
    src.balance -= amount;
    dst.balance += amount;
}
```

```
error[E0594]: cannot assign to `src.balance`, which is
behind a `&` reference
```

& is read-only.

Part 2: naive port

```
fn transfer(src: &mut Account, dst: &mut Account, ..) { .. }
transfer(&mut accounts[0], &mut accounts[1], 30);
```

Part 2: naive port

```
fn transfer(src: &mut Account, dst: &mut Account, ..) { .. }
transfer(&mut accounts[0], &mut accounts[1], 30);
```

```
error[E0499]: cannot borrow `accounts` as mutable more than once at a time
```

One `&mut` into `accounts` at a time.

Part 2: the fix

```
fn transfer(src: &mut Account, dst: &mut Account, ..) { .. }
let mut accounts = vec![ .. ];
let [src, dst] = accounts.get_disjoint_mut([0, 1]).unwrap();
transfer(src, dst, 30);
```

Two separate borrows: `get_disjoint_mut`.

Part 2: cloning and aliasing

what could go wrong

CLONE IT

```
transfer(&mut accounts[0].clone(), &mut accounts[1], 30);
```

```
alice          100
bob             80
```

Money appears.

SAME ACCOUNT TWICE

```
transfer(&mut accounts[0], &mut accounts[0], 30);
```

```
error[E0499]: cannot borrow `accounts` as mutable more than once at a time
```

No changing while borrowed.

Part 3: naive port

```
let top = richest(&accounts);
accounts.push(Account::new("erin", 500));
println!("richest before erin joined: {}", top.owner);
```

Part 3: naive port

```
let top = richest(&accounts);
accounts.push(Account::new("erin", 500));
println!("richest before erin joined: {}", top.owner);
```

```
error[E0502]: cannot borrow `accounts` as mutable because
it is also borrowed as immutable
```

`top` is still used after `push`.

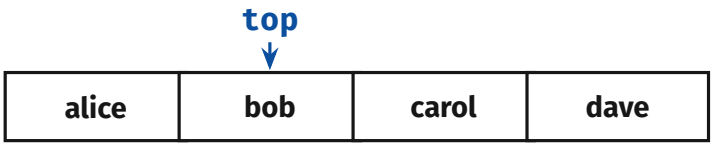
Part 3: the fix

```
let top = richest(&accounts).owner.clone();
accounts.push(Account::new("erin", 500));
println!("richest before erin joined: {top}");
```

Copy what you need first.

Part 3: why push is a problem

what could go wrong



`top` points into the `Vec`'s memory.

Part 3: why push is a problem

what could go wrong



`push`: bigger buffer, everything moves,
the old buffer is freed.



`top` would dangle.

Part 3: why push is a problem

what could go wrong



`push`: bigger buffer, everything moves,
the old buffer is freed.



`top` would dangle.

Python can't dangle. Rust refuses.

No changing a collection while iterating it.

Part 4: naive port

```
fn close_empty(accounts: &mut Vec<Account>) {
    for (i, a) in accounts.iter().enumerate() {
        if a.balance == 0 {
            accounts.remove(i);
        }
    }
}
```

Part 4: naive port

```
fn close_empty(accounts: &mut Vec<Account>) {
    for (i, a) in accounts.iter().enumerate() {
        if a.balance == 0 {
            accounts.remove(i);
        }
    }
}
```

```
error[E0502]: cannot borrow `*accounts` as mutable because
it is also borrowed as immutable
```

The loop borrows `accounts`.

Part 4: the fix

```
fn close_empty(accounts: &mut Vec<Account>) {
    accounts.retain(|a| a.balance != 0);
}
```

`retain`: no loop, no bug.

Part 4: Python, dave survives

what could go wrong

```
def close_empty(accounts):
    for a in accounts:
        if a.balance == 0:
            accounts.remove(a)
```

Part 4: Python, dave survives

what could go wrong

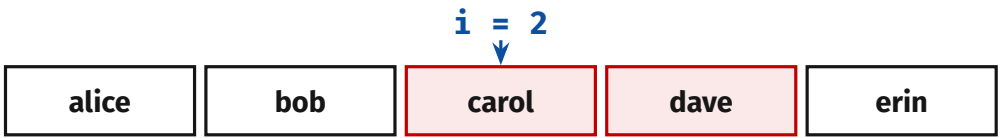
```
def close_empty(accounts):
    for a in accounts:
        if a.balance == 0:
            accounts.remove(a)
```

PYTHON3 BANK.PY

```
After closing empty accounts:
  alice          70
  bob            80
  dave            0
  erin          500
```

Part 4: why Python skips dave

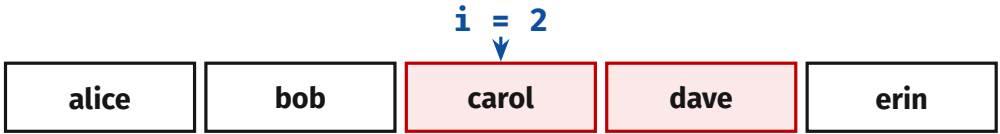
what could go wrong



remove carol: later accounts shift left.

Part 4: why Python skips dave

what could go wrong



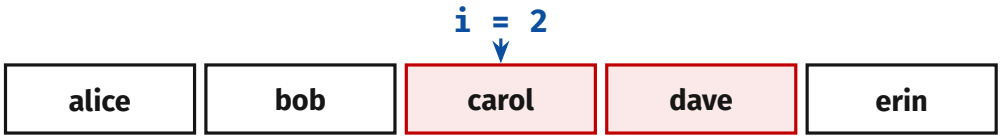
remove carol: later accounts shift left.



index 3 next. dave is at index 2: skipped.

Part 4: why Python skips dave

what could go wrong



remove carol: later accounts shift left.



index 3 next. dave is at index 2: skipped.

dave stays. No warning.

Part 4: same bug, three languages

what could go wrong

Python runs but skips an account

Part 4: same bug, three languages

what could go wrong

Python runs but skips an account

Java throws `ConcurrentModificationException`

Part 4: same bug, three languages

what could go wrong

Python runs but skips an account

Java throws `ConcurrentModificationException`

Rust does not compile

```
error[E0502]: cannot borrow `*accounts` as mutable because
it is also borrowed as immutable
```

Shared *and* mutable is forbidden.

Part 5: naive port

```
let mut alice_wallet = vec![&mut joint, &mut alice_personal];
let mut bob_wallet = vec![&mut joint, &mut bob_personal];
```

Part 5: naive port

```
let mut alice_wallet = vec![&mut joint, &mut alice_personal];
let mut bob_wallet = vec![&mut joint, &mut bob_personal];
```

```
error[E0499]: cannot borrow `joint` as mutable more than
once at a time
```

Two wallets, one `joint`.

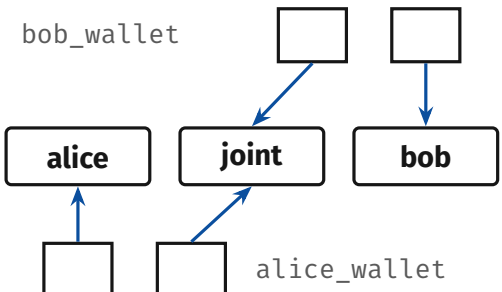
Part 5: the fix

```
let mut bank = Bank { accounts: Vec::new() };
let joint = bank.open("joint", 0);
let alice_wallet = [joint, alice_personal];
let [j, p] = bank.accounts.get_disjoint_mut(alice_wallet)
    .unwrap();
```

One owner (Bank), wallets hold IDs.

Part 5: one owner + IDs

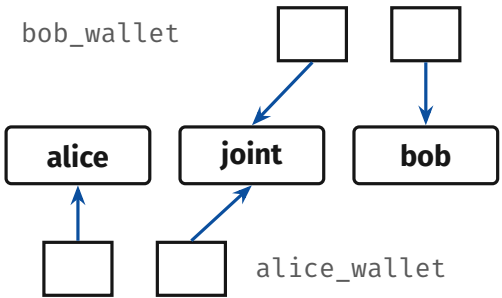
Python: shared objects



Both wallets point to the same `joint`.

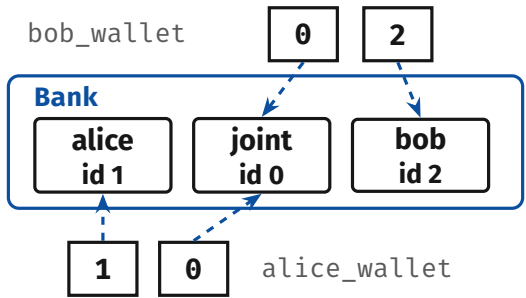
Part 5: one owner + IDs

Python: shared objects



Both wallets point to the same `joint`.

Rust: one owner, IDs



The bank owns every account.
Wallets hold IDs.

Part 5: the Python model

what could go wrong

```
let joint = shared(Account::new("joint", 0));
let alice_wallet =
    vec![Rc::clone(&joint), Rc::clone(&alice_personal)];
transfer(&mut p.borrow_mut(), &mut j.borrow_mut(), 60);
println!("joint has {} owners", Rc::strong_count(&joint));
```

```
joint has 3 owners
```

Rc<RefCell>: shared and mutable.

Part 5: checked at run time

what could go wrong

	compile time	run time
shared	`&T`	`.borrow()`
exclusive	`&mut T`	`.borrow_mut()`

```
let mine = p.borrow();
if mine.balance >= 20 {
    transfer(&mut p.borrow_mut(), &mut j.borrow_mut(), 20);
}
```

Part 5: checked at run time

what could go wrong

	compile time	run time
shared	`&T`	`.borrow()`
exclusive	`&mut T`	`.borrow_mut()`

```
let mine = p.borrow();
if mine.balance >= 20 {
    transfer(&mut p.borrow_mut(), &mut j.borrow_mut(), 20);
}
```

panic: RefCell already borrowed

Rust said no five times

	Rust enforces	Python
`print_report`	one owner	✔ fine
`transfer`	one `&mut`	✔ fine
`richest`	no change while borrowed	✔ fine
`close_empty`	no change while iterating	✖ skips dave
joint account	shared *or* mutable	✔ fine

Rust said no five times

	Rust enforces	Python
`print_report`	one owner	✔ fine
`transfer`	one `&mut`	✔ fine
`richest`	no change while borrowed	✔ fine
`close_empty`	no change while iterating	✖ skips dave
joint account	shared *or* mutable	✔ fine

> Rust rejects what it can't *prove* safe. Sometimes that's a real bug.

Ownership and borrowing rules

1. Each value has one owner.
2. Borrow with any number of `&T` or one `&mut T`.
3. A reference can't outlive the value it refers to.

> Porting to Rust helped us find a bug in the Python code (dave). Sometimes Rust is just being careful (`push`).

optional A

Threads can only share what is safe to share.

Threads: naive port

optional A

```
clerks.push(thread::spawn(|| payroll(&mut company)));
```

Threads: naive port

optional A

```
clerks.push(thread::spawn(|| payroll(&mut company)));
```

```
error[E0373]: closure may outlive the current function, but it
borrows `company`, which is owned by the current function
```

A thread may outlive `main`.

Threads: naive port

optional A

RUSTC SUGGESTS MOVE

```
clerks.push(thread::spawn(move || payroll(&mut company)));
```

```
error[E0382]: borrow of moved value: `company`
```

`move` gives the first thread the only copy.

Threads: the fix

optional A

```
let company = Arc::new(Mutex::new(company));
for _ in 0..4 {
    let company = Arc::clone(&company);
    clerks.push(thread::spawn(move || payroll(&company)));
}
```

```
company balance: 4000 (expected 4000)
```

`Arc` shares, `Mutex` guards.

Threads: the fix

optional A

```
let company = Arc::new(Mutex::new(company));
for _ in 0..4 {
    let company = Arc::clone(&company);
    clerks.push(thread::spawn(move || payroll(&company)));
}
```

```
company balance: 4000 (expected 4000)
```

Arc shares, Mutex guards.

RC INSTEAD OF ARC

```
error[E0277]: `Rc<std::sync::Mutex<Account>>` cannot be sent
between threads safely
```

Threads: Python loses deposits

what could go wrong, optional A

```
def payroll(account):
    for _ in range(1000):
        b = account.balance
        time.sleep(0)
        account.balance = b + 1
```

Threads: Python loses deposits

what could go wrong, optional A

```
def payroll(account):
    for _ in range(1000):
        b = account.balance
        time.sleep(0)
        account.balance = b + 1
```

PYTHON3 BANK.PY (THE NUMBER CHANGES ON EVERY RUN)

```
company balance: 1038 (expected 4000)
```

Will this compile?


```
fn main() {
    let n = 5;
    let m = n;
    println!("{n}");
}
```

Compiles? ✋

Will this compile?


```
fn main() {
    let n = 5;
    let m = n;
    println!("{n}");
}
```

✔ Compiles

`i32` implements `Copy`. The assignment copies the number, so `n` is still usable.

Will this compile?


```
struct Account { owner: String }

fn main() {
    let mut accounts = vec![Account { owner: "alice".into() }];
    let first = &accounts[0];
    println!("{}", first.owner);
    accounts.push(Account { owner: "bob".into() });
}
```

Compiles? ✋

Will this compile?

optional B, 2 / 3

```
struct Account { owner: String }

fn main() {
    let mut accounts = vec![Account { owner: "alice".into() }];
    let first = &accounts[0];
    println!("{}", first.owner);
    accounts.push(Account { owner: "bob".into() });
}
```

✔ Compiles

We last use `first` before `push`, so the borrow has ended.

Will this compile?

optional B, 3 / 3

```
struct Account { owner: String }

fn open() -> &Account {
    let a = Account { owner: "alice".into() };
    &a
}

fn main() {
    println!("{}", open().owner);
}
```

Compiles? ✋

Will this compile?

optional B, 3 / 3

```
struct Account { owner: String }

fn open() -> &Account {
    let a = Account { owner: "alice".into() };
    &a
}

fn main() {
    println!("{}", open().owner);
}
```

✕ **error[E0106]: missing lifetime specifier**

a is dropped when open returns. Adding `'static` won't fix this and gives E0515.

Questions?

- The Rust Book, chapter 4: https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html
- Rust Playground: https://play.rust-lang.org/
- Materials: github.com/thecodinglab/rust-ownership-tutorial